Metaverse Framework for Wireless Systems Management

Ilias Chrysovergis, Alexandros-Apostolos A. Boulogeorgos, *Senior Member, IEEE*, Theodoros A. Tsiftsis, *Senior Member, IEEE*, and Dusit Niyato, *Fellow, IEEE*

Abstract

This article introduces a metaverse framework designed for simulation, emulation, and interaction with wireless systems. The framework integrates extended reality (XR), digital twins (DTs), artificial intelligence (AI), the internet of things (IoT), blockchain, and advanced 5G/6G networking solutions to create a platform for both system development and management. Using XR, users can interact with complex systems, while DTs enable real-time monitoring and optimization. AI improves system performance, and IoT devices provide real-time sensor data to increase simulation accuracy. In addition, blockchain ensures secure, decentralized interactions, and wireless networks offer the infrastructure for uninterrupted communication. This framework serves as a tool for exploring, developing, and optimizing wireless systems, with the aim of offering a significant understanding into the future of networked environments. The framework achieves sub-10 ms decision-to-action latency through integrated operation of all six layers, demonstrated via a UAV positioning use case that delivers <1 ms URLLC connectivity, sub-10 ms AI inference, 12 ms ray-tracing updates, and 3–6 s blockchain finality with audit trails, while the RL-optimized positioning policy significantly improves network coverage for disadvantaged users.

I. Introduction

The rapid advancements in extended reality (XR), digital twins (DT), artificial intelligence (AI), internet of things (IoT), blockchain, and networking have paved the way for experts in various industries to envision intelligent, physics-based, three-dimensionally-rendered, interoperable, secure, and distributed spaces, often called metaverses. These advances can address the growing complexity of next-generation wireless systems, which require more sophisticated tools for visualization, optimization, and management. The increasing scale and complexity of wireless systems pose difficulties in terms of performance optimization and security. Traditional network management approaches lack the flexibility to address the multifaceted requirements of these networks that have evolved to support AI, ultra-reliable and low-latency communications, immersive communications, Integrated Sensing and Communication (ISAC), massive communications, and ubiquitous connectivity.

By integrating DT, AI, and XR, metaverse-based systems can simulate, optimize, and visualize complex network architectures in real-time. Operators can predict and mitigate network congestion, simulate failure scenarios, and optimize network coverage in highly dynamic environments. The upcoming demands of next-generation applications, such as autonomous vehicles, smart cities, and industrial automation, require systems that are performant, adaptive, secure, and resilient. The metaverse bridges the gap between the physical and digital worlds, enabling real-time monitoring and autonomous decision-making, transforming next-generation wireless networks from reactive to proactive management paradigms.

Motivated by the above observations, Khan et al. [2] explored the potential of the metaverse in the advancement of sixth-generation (6G) wireless systems. They presented a metaverse that serves as a virtual platform for network designers, developers, and engineers to analyze, optimize, and operate wireless networks. Lotfi et al. [1] addressed the issue of incentivizing IoT devices to share sensing data essential for creating accurate DTs within the metaverse. Hashash et al. [3] introduced a framework that categorizes the metaverse experience into seven worlds: enhanced reality, DTs, mirror worlds, lifelogging, virtual worlds, augmented reality, and mixed reality. Sehad et al. [4] presented a framework that explores Generative AI (GenAI) to enhance multisensory experiences in next-generation systems. Khan et al. [5] presented a framework that uses DTs to enable intelligent and self-sustaining 6G systems. Decentralized physical infrastructure networks (DePIN) leverage blockchain technology to decentralize the control and management of physical devices, addressing the limitations of centralized infrastructure through collective ownership, reduced operating costs, and improved privacy and security [13].

In [6], Open-Radio Access Network (O-RAN) ALLIANCE's next Generation Research Group (nGRG) explored the integration of distributed applications in RAN environments, establishing critical use cases and technical requirements that enable metaverse applications that require ultra-low latency and

Ilias Chrysovergis is with the Department of Informatics & Telecommunications, University of Thessaly, 35100 Lamia, Greece, and also with Metatopia L.P., Thessaloniki, 54624, Greece; Alexandros-Apostolos A. Boulogeorgos is with the Department of Electrical and Computer Engineering, Democritus University of Thrace, Xanthi, 67100, Greece; Theodoros A. Tsiftsis (corresponding author) is with the Department of Informatics & Telecommunications, University of Thessaly, Lamia, 35100, Greece (e-mail: tsiftsis@uth.gr); Dusit Niyato is with the College of Computing & Data Science (CCDS), Nanyang Technological University, 639798, Singapore.

The work of Theodoros A. Tsiftsis was supported by the Project NEURONAS. The Research Project NEURONAS is implemented in the framework of H.F.R.I Call "3rd Call for H.F.R.I.'s Research Projects to Support Faculty Members & Researchers" (H.F.R.I. Project Number: 25726).

Digital Object Identifier: 10.1109/MIOT.2026.3680843
Date of Current Version: 25 August 2026
Date of Publication: 13 May 2026

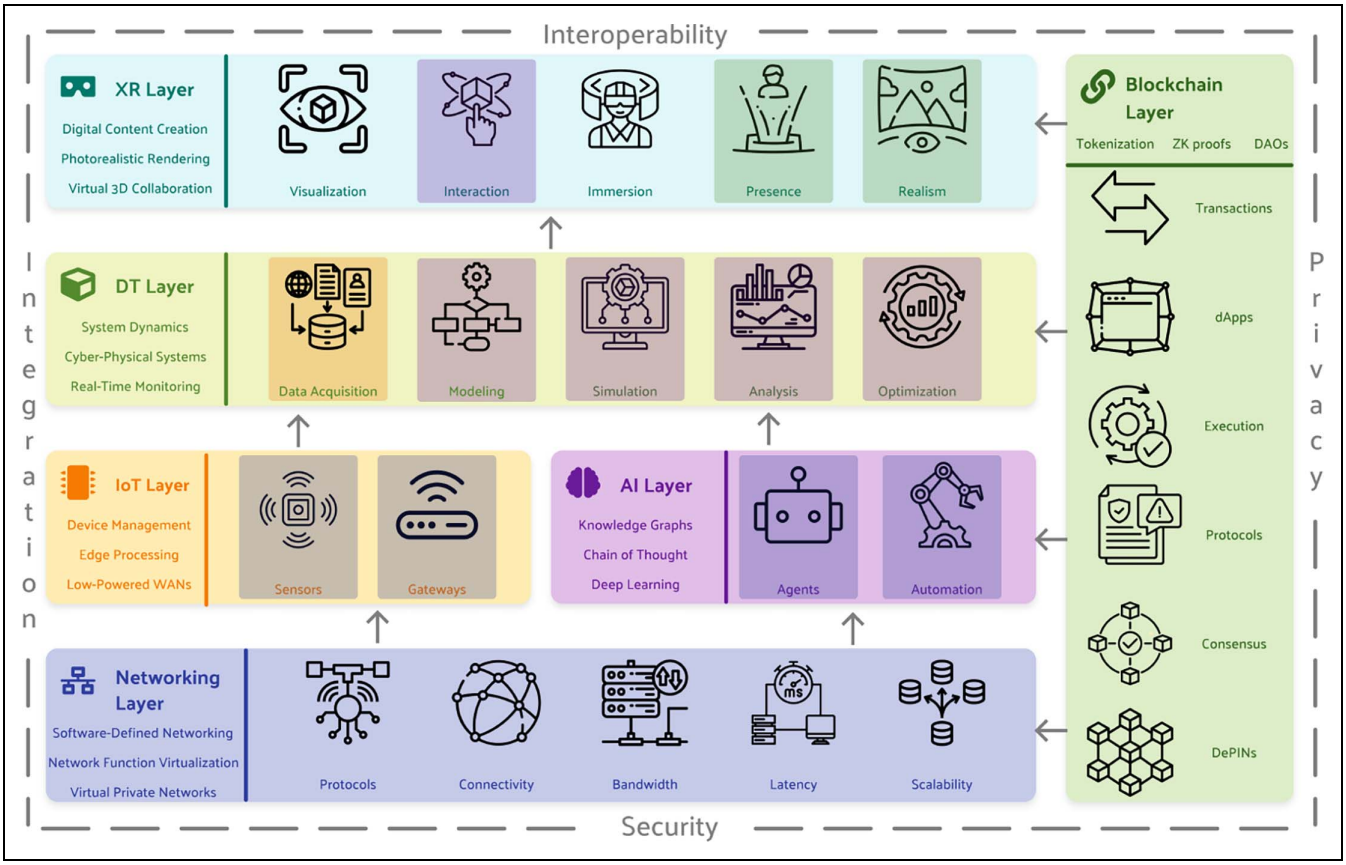

FIG. 1. The framework leverages a networking layer, alongside IoT and AI, to feed a DT layer. The results are rendered via an XR layer for immersive interaction. A vertical blockchain layer ensures trust, security, and decentralization across the system. Key principles of security, privacy, and interoperability are enforced across all layers to ensure a robust and integrated system.

high reliability. Polese et al. [7] introduced Colosseum, a large-scale wireless network emulator that serves as a DT for O-RAN systems and demonstrated how Colosseum facilitates the testing of complex scenarios involving multiple base stations, user devices, and network slices. Jiang et al. [8] articulated an approach using neural representations to create learnable wireless DTs. Alikhani et al. [9] introduced the large wireless model (LWM), a foundation model designed for the prediction and characterization of wireless channels. Yang et al. [10] reported a novel framework that uses diffusion models to address complex network optimizations in IoT environments. Zou et al. [11] introduced TelecomGPT, a framework designed to transform general-purpose LLMs into telecom-specific models.

Although the above studies established the foundation, they focus on specific facets of the wireless metaverse. Either they explore these technologies in isolation or propose high-level concepts without detailing a concrete structure. Thus, a gap remains in the literature: the lack of an integrated framework that combines all key enablers into a unified framework designed for the end-to-end management of those systems. Motivated by this, our work introduces a novel framework that integrates these disparate technologies, spanning from the physical infrastructure to the immersive user interface, and aims to provide a complete blueprint for the next generation of network management tools.

The contributions of the paper are summarized as follows: (i) We propose a metaverse-based framework that spans from the physical to the application layer and enables simulation, emulation, and interaction with complex wireless systems. *Unlike prior studies that typically focus on these technologies in isolation or present high-level concepts, our work provides a concrete, end-to-end architecture.* (ii) We devise a blockchain architecture to make virtual worlds secure, interoperable, and decentralized. *This addresses the limitations of centralized platforms by providing a layer of trust and eliminating single points of failure.* (iii) We demonstrate the practical application and effectiveness of the framework through an aerial network use case. *We move beyond theoretical proposals by providing empirical evidence of the framework's practical capabilities.*

II. Metaverse Framework

Fig. 1 shows the framework's technology stack that spans from the network infrastructure to the XR user interface. Each layer has a specific role, outlined as follows.

Networking Layer: This layer provides the foundation for connectivity across the system. Key concepts include Software-Defined Networking (SDN), Network Function Virtualization (NFV), and Virtual Private Networks (VPNs). This layer also includes edge computing, which deploys resources at the edge of the network to improve efficiency. Attributes such as protocols, connectivity, bandwidth, latency, and scalability determine the efficiency with which the system functions.

IoT Layer: The IoT layer comprises physical sensors, gateways, and edge processing components.

Wireless access networks and device management are techniques to handle large-scale deployments. The data collected from real-world entities flow upward in the framework's stack for processing.

AI Layer: Processes data streams and uses knowledge graphs, chain-of-thought reasoning, and deep learning to support decision-making. *Agents* autonomously perform tasks, while *automation* extends the capabilities of these agents' to real-world control. The AI layer adds cognitive and analytical depth, transforming raw sensor signals into actions and insights that lead to optimal solutions.

DT Layer: DTs create virtual replicas of cyber-physical systems, to enable the modeling of system dynamics and real-time monitoring. The DT layer consists of the following components: (i) *Data acquisition*, which involves integration with IoT devices to collect real-time data from the system, (ii) *modeling and simulation*, which includes algorithms to create and update digital replicas, and (iii) *analysis and optimization*, which provide tools for analyzing system performance and optimizing operations.

XR Layer: It acts as an immersive interface that allows users to interact with the system. The technologies that enable this layer are digital content creation, photorealistic rendering, and collaborative virtual environments. It supports (i) *3D visualization*, which is used for real-time rendering of the system components and their relations, (ii) *interactive controls* composed by interfaces for manipulating components, and (iii) *simulation scenarios* comprised of customizable use cases for testing.

Blockchain Layer: Ensures trust and secure transactions across the framework. Methods like tokenization, zero-knowledge proofs, and decentralized autonomous organizations (DAOs) belong to this layer. Provides mechanisms for record-keeping, consensus, and privacy. Those features are necessary when data are exchanged among untrusted entities. Key components include: (i) *Security*, which leverages cryptography to ensure all operations are recorded immutably on the distributed ledger, (ii) *interoperability*, which uses smart contracts to enable data sharing across networks, and (iii) *decentralization*, which ensures that the system cannot be controlled by any single entity.

Cross-Cutting Aspects: The framework ensures integration and interoperability among all layers by using standardized interfaces and data models. Security and privacy are enforced at all levels to protect system resilience by implementing mechanisms to protect data integrity, confidentiality, and availability. This includes: (i) *Authentication and authorization*, which employs JSON web tokens, (ii) *data encryption*, which ensures protection in transit and at rest, (iii) *access control*, which restricts resource access based on permissions, and (iv) *password-less systems*, which utilize biometrics, email or SMS-based one-time codes, or hardware security keys.

III. Blockchain-Based Decentralized Architecture

Blockchain technology enables a secure, interoperable, and decentralized metaverse ecosystem. The blockchain enables the establishment of a token-based economy, where users can obtain digital assets (e.g., virtual equipment), data sets, services (e.g., custom-built simulations), and resources.

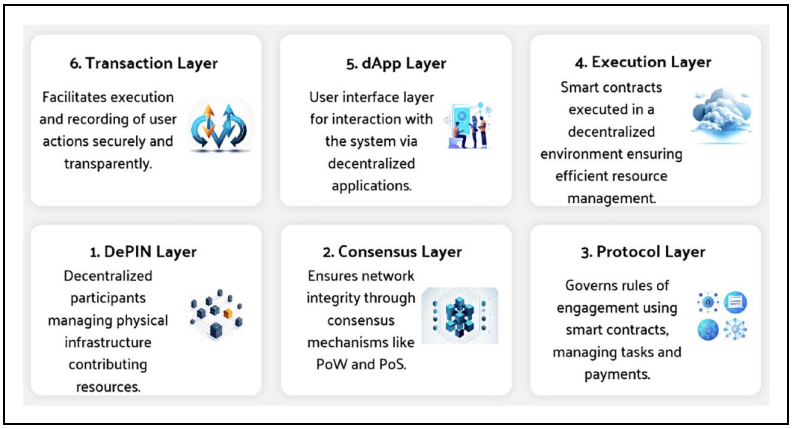

FIG. 2. Blockchain-based decentralized architecture's layers.

This economy ensures that all transactions are tamper-proof. Users can purchase tokens that allow them to access virtual products and services. These tokenization capabilities provide a framework for managing ownership and access to these digital assets and services, allowing users to buy, sell, or lease them securely within the marketplace. For example, a company could tokenize a custom-built DT of a real communication system and offer it to other companies for testing in exchange for tokens.

In addition, blockchain enables automation through the use of smart contracts. These self-executing contracts ensure that once a user pays for a digital asset, simulation, or resource, the required service is automatically allocated. Smart contracts automatically enforce the terms of the transaction, such as usage limits or access periods. This automation allows users to focus on system optimization and experimentation while the blockchain handles resource distribution.

(a) Architecture: The architecture (Fig. 2) leverages decentralization at every step, from physical infrastructure to final user interaction. The sublayers are given below.

DePIN Sublayer: The architecture begins with the DePIN Sublayer, where participants manage the physical infrastructure that contributes data or services. These participants operate nodes on the network, providing the resources that power decentralized applications (dApps). The contribution of infrastructure is recorded and validated, ensuring decentralization, transparency, and rewards for node operators.

Consensus Sublayer: Moving up to the Consensus Sublayer, the architecture employs a decentralized ledger that uses consensus mechanisms to validate transactions and ensure network integrity. Each transaction is validated through this consensus mechanism. This layer ensures that all nodes on the network agree on the state of the system, ensuring the security of each service request.

Protocol Sublayer: In this sublayer, smart contracts manage how tasks are assigned to DePINs, handle payment through escrow, and ensure that the nodes comply with agreed-upon terms. These smart contracts automate processes such as assignment of computation tasks, payment distribution, and validation of results. Additionally, in cases where a DAO is in place, token holders can collectively manage the governance decisions of the system through this layer.

Execution Sublayer: In this sublayer, the smart contracts are executed, and the actual computation

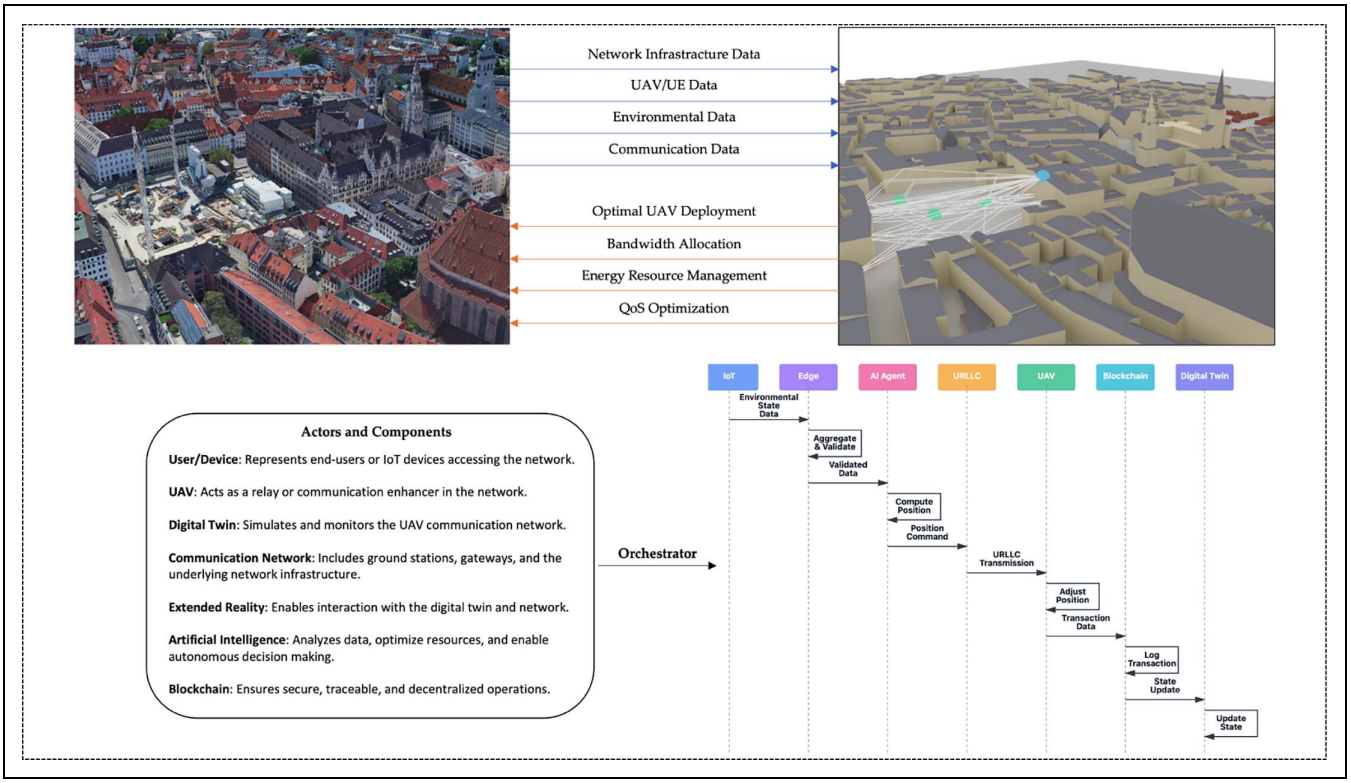

FIG. 3. A metaverse-enabled aerial network demonstrating the integration of all six architectural layers.

takes place. The selected DePIN performs the task requested by the user, and the results are submitted to the blockchain. During execution, gas costs manage resource consumption, ensuring the network's efficiency and preventing abuse.

dApp Sublayer: In this sublayer, users interact with the system through a front-end interface, connecting them to the blockchain through wallets and smart contracts. The dApp provides an environment where the computation request is initiated, the result is displayed, and transactions are handled.

Transaction Sublayer It facilitates the execution and recording of user actions, such as submitting tokens or provisioning services. When a user requests a digital asset or service via the dApp, the transaction is broadcasted to the blockchain, validated, executed by a DePIN node, and finalized. The transaction moves through all layers and concludes when the result and payment are confirmed on the blockchain.

(b) Decentralized computation flow: The process begins at the transaction sublayer, where a user requests a virtual asset or service via a decentralized application (dApp). This request is signed and transmitted to the blockchain as a transaction. At the consensus layer, it is validated using a consensus mechanism, ensuring that it meets all requirements. Following validation, a DePIN node is selected to carry-out the task. The task is then managed by the protocol layer, where smart contracts handle the assignment, escrow the payment, and ensure that the DePIN adheres to the requirements.

Once the DePIN node has completed the task, it submits the results to the blockchain through the execution layer. The protocol layer verifies the accuracy of the task and, if all conditions are met, releases the payment to the DePIN node. The transaction is then finalized in the consensus layer, ensuring that both the result and payment are permanently recorded. Finally, the user is notified of the result through the dApp, receiving the outcome of the task.

IV. Metaverse-Enabled Aerial Network

To validate the proposed metaverse framework, this section presents a simulated use case that demonstrates the integration of all six architectural layers for aerial networks.

A. System Architecture and Interaction Flow

Fig. 3 presents the architecture and operational flow of an autonomous UAV positioning optimization system, demonstrating the integration of all six framework layers. The system achieves sub-10 ms decision-to-action latency[1] while maintaining blockchain-verified audit trails, operating autonomously with human oversight provided through immersive spatial interfaces. The architecture implements a dual-path design that separates real-time control from blockchain validation.

Networking Layer: The layer provides connectivity through three 5G network slices operating in parallel: (i) URLLC slice delivering sub-1ms latency for UAV control with 99.999% reliability (achieved via 5G NR with 60 kHz subcarrier spacing and mini-slot scheduling), enabling immediate command execution, (ii) eMBB slice supporting 100-200 Mbps throughput for XR streaming (utilizing 100 MHz bandwidth at 3.5 GHz with 256-QAM and 4×4 MIMO, consistent with commercial 5G deployments), and (iii) mMTC slice connecting 10,000+ IoT devices per km^2 for environmental monitoring (based on NB-IoT and massive MIMO capabilities). An SDN controller manages traffic flows with sub-100 ms routing updates. MEC servers co-located with base stations reduce processing

[1] This latency is achieved through: (i) MEC deployment co-located with base stations eliminating cloud round-trip delays, (ii) GPU-accelerated AI inference, (iii) pre-computed ray-tracing lookup tables reducing DT update time, and (iv) parallel processing with URLLC slice.

latency by operating computations on the edge instead of the cloud.

IoT Layer: A network of sensors provides environmental ground truth: LoRaWAN weather stations, NB-IoT RF spectrum monitors that detect interference patterns, and 5G-enabled user equipment that report real-time signal-to-interference-plus-noise ratio (SINR) measurements. Edge gateways perform Next Generation Service Interfaces - Linked Data (NGSI-LD) semantic aggregation, achieving data reduction while preserving information fidelity. All sensor data includes Elliptic Curve Digital Signature Algorithm (ECDSA) cryptographic signatures verified automatically at edge nodes before propagation.

AI Layer: A lightweight proximal policy optimization (PPO) agent (termed Scene-Aware PPO, or SA-PPO; see Section IV-C) enables ultra-fast autonomous decision-making. Deployed on GPUs at MEC nodes, the neural network processes state vectors and generates optimal positioning actions. The agent performs real-time inference with sub-10 ms latency through mixed-precision computation, while policy updates occur offline during training phases to continuously improve performance based on experience.

Digital Twin Layer: A ray-tracing engine maintains a virtual replica of the urban scene. The IoT sensor data calibrate the properties of the materials and atmospheric effects, enabling the digital twin to compute propagation paths and generate channel matrices for all users.

Blockchain Layer: A distributed ledger with N validator nodes ensures a trustless operation through consensus mechanisms. Validators run PoS/dBFT consensus [14], achieving transaction finality in 3-6 seconds based on stake-weighted selection, where nodes with higher stake, uptime, and reputation have greater probability of block production. Smart contracts programmatically enforce business logic, rejecting positions with SINR below -15 dB[2], verifying ECDSA cryptographic signatures from sensors, requiring $2N/3 + 1$ validator signatures for position updates, and emitting optimization events when SINR drops below -10 dB.

XR Layer: An immersive visualization system renders the digital twin and blockchain state in a 3D environment. Operators wearing XR headsets or smart glasses gain awareness through volumetric SINR heat maps, real-time UAV trajectory visualization with historical paths, and performance dashboards. Operators can interact through natural gestures to inspect detailed metrics or perform emergency stop gestures. Operators observe the AI's decisions, watch blockchain validations propagate, and gain understanding of system behavior.

Critical Path Separation: The architecture implements separation between real-time control and trust mechanisms. When the AI agent computes a new position, the command is immediately transmitted via URLLC to the UAV, which begins to move within 6-7 ms of the decision. Similarly, the position update enters the blockchain pipeline for asynchronous validation, creating an immutable audit trail without impeding real-time operation.

The complete control cycle demonstrates the framework's capability for intelligent, autonomous wireless network management: IoT sensors report environmental state → Edge gateways aggregate and validate data → AI agent computes optimal position → Command transmits via URLLC → UAV adjusts position → Blockchain logs transaction→ Digital twin updates based on new state.

Algorithm 1: Multi-Layer System Integration

```
Initialize: Create layers                                   ▷ Orchestrator
while simulation active do                                  ▷ Orchestrator
    // Phase 1: Parallel collection
    Collect sensor data from IoT devices                    ▷ IoT Layer
    Manage network slices                                   ▷ Networking Layer
    Process blockchain transactions                         ▷ Blockchain Layer
    Await all parallel operations complete                  ▷ Orchestrator
    // Phase 2: Critical path
    Update DT with sensor calibration                       ▷ DT Layer
    Compute ray-tracing                                     ▷ DT Layer
    Extract observation vector                              ▷ DT Layer
    AI inference with PPO forward pass                      ▷ AI Layer
    Transmit action                                         ▷ Networking Layer
    // Phase 3: Finalization
    Log position with multi-sig                             ▷ Blockchain Layer
    Transaction finality                                    ▷ Blockchain Layer
    Render next XR frame                                    ▷ XR Layer
end while
Output: Complete simulation traces with UAV trajectory,
    SINR measurements, network metrics, and blockchain
    audit trail                                             ▷ Orchestrator
```

B. Simulation Implementation and System Integration

Orchestrator: During each simulation cycle (Alg. 1), the system executes in phases with different concurrency models. First, the parallel execution phase handles sensor data collection, network slice management, and blockchain transaction processing concurrently, with all three completed before proceeding. Second, after the parallel phase completes, the sequential critical path executes within a 6-millisecond deadline, updating the digital twin environment, computing radio propagation paths, extracting observations, performing 5-millisecond AI inference, and transmitting commands via URLLC. Simultaneously with the critical path, the asynchronous logging path records position updates to the blockchain operating with 3-6 second finality. Finally, visualization updates render frames based on the state of the environment.

Networking Implementation: The networking layer simulates 5G network slicing with three service profiles, each optimized for different application requirements. Each slice receives dedicated bandwidth allocation: URLLC (10 Mbps), eMBB (127 Mbps), and mMTC (5 Mbps). Bandwidth isolation is enforced by calculating transmission time from the selected slice's allocated bandwidth: $T_{\text{tx}} = (D \times 8)/\, B_{\text{slice}} \times 1000$ ms, where B_{slice} is the fixed bandwidth parameter, the factor of 8 converts bytes to bits, and D is the data size in megabytes. This ensures that traffic in one slice does not impact transmission times in other slices. Additionally, each slice enforces specific latency targets appropriate to its service class.

When packets arrive, the system performs packet inspection to classify each one in the appropriate slice. URLLC packets receive priority treatment. The system calculates transmission delay based on packet size and available bandwidth, then verifies that the

[2] The −15 dB threshold represents the minimum SINR for reliable 5G NR QPSK modulation

Algorithm 2: DT-aided RL-based UAV Positioning for Throughput Maximization

1: **Initializion Phase:** Create the simulation environment. Define UAV 'Transmitter' and 'Receiver' devices with initial positions. Instantiate the PPO agent with a multi-layer perceptron (MLP) policy.
2: **for** each training episode **do**
3: Reset the environment, placing the UAV and receivers at their starting locations.
4: **for** each step t in the episode **do**
5: Observe the current state s_t (UAV's 3D position).
6: The PPO agent selects an action a_t (a discrete movement vector) based on its current policy $\pi(a_t \mid s_t)$.
7: Apply the action to the UAV, updating its position in the simulation.
8: Use the DT's ray-tracer to compute the channel impulse responses between the UAV and receivers.
9: Calculate the SINR for each receiver based on the channel conditions.
10: Compute the reward r_t as the sum of all receiver SINRs.
11: The PPO agent stores the transition (s_t, a_t, r_t, s_{t+1}).
12: Update the agent's policy using the collected transition data to maximize the expected cumulative reward.
13: **end for**
14: **end for**
15: **Output:** A trained PPO agent with an optimized policy for UAV positioning.

delay remains below the 1-millisecond threshold. For mMTC packets the system implements Non-Orthogonal Multiple Access techniques to accommodate the massive number of concurrent connections while minimizing signaling overhead. eMBB packets undergo adaptive modulation and coding scheme selection based on current channel quality, optimizing throughput while maintaining quality of service.

The networking layer achieved validated performance metrics with URLLC latency averaging 0.8 milliseconds with 0.1-millisecond standard deviation. eMBB throughput averaged 127 Mbps with 15-millisecond latency, while mMTC successfully supported 12,000 concurrent IoT connections per square kilometer.

IoT Sensor Network: The IoT layer simulates 180 heterogeneous sensors with realistic data generation patterns. Each sensor type implements domain-specific measurement processes that capture the complexity of real-world deployments [15].

Weather stations generate temporally correlated temperature measurements using a first-order autoregressive process. Each station maintains its previous temperature reading and applies a Gaussian random walk with damping to simulate realistic thermal inertia. The actual temperature evolves gradually based on a normally distributed change scaled by 0.1 to prevent unrealistic jumps. Measurement noise, modeled as Gaussian with a standard deviation of 0.1-degree Celsius, simulates sensor imperfections. Humidity measurements use Kriging interpolation across a spatial grid, capturing geographic correlation patterns. Wind measurements follow a Gauss-Markov process based on historical readings, modeling the autocorrelated nature of wind patterns.

RF spectrum monitors implement software-defined radio behavior, scanning both the 2.4 and 5 GHz bands with 20 MHz resolution. The scanning process detects interference signals above a -80 dBm threshold, identifying WiFi and LTE sources that affect performance.

The user equipment simulates the behavior of mobile devices using mobility models. The Random Waypoint model governs pedestrian movement patterns, while Gauss-Markov processes model vehicular trajectories. Each user equipment continuously measures SINR from its serving cell and computes achievable throughput using Shannon capacity formulas.

AI Agent: The agent's goal is to identify the best possible location for the UAV that maximizes the sum of the SINR for all ground receivers, which correlates with the overall network throughput. The problem is framed as an RL task, where an intelligent agent learns to make sequential decisions in a simulated environment to maximize a cumulative reward.

The starting positions of the UAV and the receivers are chosen to represent a realistic urban scenario where the receivers are spread out on the ground (at a height of 1.5 meters, typical for user devices) and the UAV starts in a position that requires optimization to improve signal quality. The state observed by the agent at each step is the current 3D coordinates of the UAV. The action space is discrete, allowing the agent to move the UAV incrementally along the x, y, or z axes. Following each action, the environment calculates the resulting SINR for each receiver, and the sum of these values is returned to the agent as a reward signal, guiding its learning process toward optimal positioning strategies. During the training phase, the agent interacts with the DT over multiple episodes, iteratively refining its policy through trial-and-error to achieve convergence on high-reward states that correspond to superior UAV positions. The entire training workflow is detailed in Algorithm 2. Using sum-SINR rather than sum-capacity (which applies $\log_2(1+\cdot)$) provides better reward shaping for disadvantaged receivers, since low-SINR users experience proportionally larger changes in SINR than in capacity. This choice implicitly prioritizes fairness across receivers: improving the weakest link (Rx1) yields larger reward increments than further boosting already-strong receivers. Since Shannon capacity is a monotonically increasing function of SINR, maximizing sum-SINR remains a valid proxy for capacity maximization. This training algorithm executes centrally on simulation infrastructure. During deployment, only the policy inference step (line 6) runs on the UAV or nearby MEC node..

The PPO agent employs an Actor-Critic architecture with separate policy and value networks, each containing three fully-connected hidden layers [256, 128, 64] neurons with ReLU activation, batch normalization, and dropout regularization ($p = 0.1$) to prevent overfitting. The policy network outputs action probabilities via softmax over six discrete actions ($\pm X$, $\pm Y$, $\pm Z$ movements), while the value network estimates state values for advantage computation. We use Adam optimizers with learning rates of $\alpha_\pi = 3 \times 10^{-4}$ (policy) and $\alpha_V = 6 \times 10^{-4}$ (value), discount factor $\gamma = 0.99$, and PPO's epsilon-clipped surrogate objective with $\epsilon = 0.2$ to ensure stable policy updates. Gradient clipping (max norm 0.5) and entropy regularization (coefficient $\beta = 0.01$) further stabilize training. The 20-dimensional state vector encodes UAV coordinates, user positions, and SINR

measurements, with returns normalized via $\hat{G}_t = (G_t - \mu)/(\sigma + 10^{-8})$ before advantage estimation $A_t = G_t - V(s_t)$.

The time complexity of the algorithm is dominated by its nested loops, running for E episodes each with T steps, resulting in a base complexity of $O(E\,T)$. In each step, key operations include state observation and action selection via a MLP policy at $O(H\,W^2)$, where H and W are the number of hidden layers of the MLP and their width, respectively, position updates at $O(1)$, channel impulse response computation at $O(R\,P\,L)$ with R being the receivers, P the paths, and L the scene complexity, SINR calculation at $O(R\,M)$, where M represents the number of multipath components, reward computation at $O(R)$, transition storage at $O(1)$, and policy updates at $O(H\,W^2)$. L quantifies the complexity of the scene, typically representing the number of geometric elements (e.g., triangles, polygons) within the simulated 3D environment. The total time complexity is $O(E\,T\,(R\,P\,L + H\,W^2))$, and RT being the main bottleneck in urban scenarios.

The space complexity is driven by the storage requirements of the simulation environment, including the 3D mesh data of the urban scene and the RT buffers in $O(L + R)$, the MLP weights of the PPO agent in $O(H \cdot W^2)$, and the transition buffers from episodes in $O(T\,D)$ where D represents the dimensionality of a single transition tuple. The total space complexity is $O(L + H\,W^2 + E\,T\,D)$, though the maximum usage usage per episode is $O(L + H\,W^2 + T\,D)$ as the transitions do not persist between episodes, with the scene complexity L typically dominating in memory-intensive urban simulations.

DT Integration: The digital twin layer creates a virtual replica of the physical environment using Sionna ray-tracing [12]. The interference map construction uses Kriging interpolation to spatially distribute RF monitor measurements across the simulation area. The transmitter positioned at the UAV location broadcasts with 23 dBm power at 3.5 GHz carrier frequency. For each of the receivers in the scene, the system computes all propagation paths up to 5 reflection orders. A single position update is completed in 12 milliseconds, enabling near-real-time digital twin updates. Path loss calculations aggregate all multipath components, applying appropriate phase shifts and power scaling based on path geometry. The generation of the SINR grid uses these path loss values combined with the calibrated noise floor to produce spatially resolved signal quality maps.

The DT generates training data through Sionna ray-tracing calibrated by 180 real IoT sensors (LoRaWAN weather stations, NB-IoT RF monitors, 5G UE). This approach is essential because: (i) it provides physically accurate electromagnetic propagation models with material properties and atmospheric attenuation impossible to replicate with simplified path loss formulas, (ii) it enables safe exploration of failure scenarios (e.g., UAV crashes and network outages) without equipment damage or service disruption, (iii) it delivers perfect ground-truth SINR measurements unattainable from noisy physical sensors that suffer from quantization errors and sampling limitations, and (iv) it allows rapid iteration across diverse urban geometries and weather conditions without deployment costs. This sensor-calibrated synthetic environment ensures the DT remains synchronized with physical conditions while providing the data quality necessary for robust policy learning.

Blockchain Implementation: Block generation begins with consensus timing simulation, drawing finality time from a uniform distribution that models the Poisson arrival process observed in generation blockchain systems. Validator selection employs stake-weighted random sampling, where each validator's weight combines their staked token amount with an uptime reputation factor.

The block building process selects transactions from the mempool based on gas price priority. Multisignature validation requires approval from multiple validators, preventing single validators from approving incorrect positions. Sensor data transactions encourage sensors to aggregate readings rather than submitting individual transactions. The smart contract verifies that the ECDSA signatures match the public keys of the registered sensor.

Byzantine agreement proceeds through a voting phase where all validators evaluate the proposed block [14]. The proposer transmits the block to all validators who independently verify the validity of the transaction and the block structure. Each validator submits a vote, either approving or rejecting the block. The consensus threshold requires at least a ceiling of two-thirds plus one vote for approval. If consensus succeeds, the block commits to the chain and metrics update to record finality time and throughput.

XR Visualization The XR layer provides immersive visualization of system state through a rendering pipeline. Each frame rendering cycle begins by recording the start time and calculating the frame time budget. The visualization update phase processes several components sequentially: SINR heatmap updates, UAV model transformations, trajectory trail rendering, and performance metric graphs. SINR heatmap updates interpolate the grid computed by the digital twin applying color mapping. The geometric transformation (translation and rotation) of the UAV model updates the aircraft's position and orientation, rendering trajectory trails to show historical movement. The rendering of metric graphs displays real-time performance graphs showing throughput, latency, SINR, and other key indicators.

C. Blockchain Security and Threat Mitigation

The blockchain layer addresses security threats across all six sublayers through integrated defense mechanisms. In the DePIN sublayer, node registration with stake deposits and reputation scoring prevent rogue sensors from joining the network and provide resistance to Sybil attacks. In the consensus sublayer, the hybrid PoS/dBFT mechanism with $\lceil 2N/3 \rceil$ validator agreement ensures Byzantine fault tolerance against malicious validators. The immutable ledger prevents tampering with historical data committed to the blockchain. At the protocol sublayer, smart contracts enforce multi-signature validation requiring $\lceil N/2 \rceil$ validators' approval for position updates to prevent unauthorized commands from compromised validators. The stake-based sensor data registry with automatic slashing penalties economically discourages the submission of malicious measurements. In the execution sublayer, the gas pricing mechanism prevents denial-of-service attacks by enforcing a transaction rate limit based on computational cost. In the dApp sublayer, wallet-based cryptographic authentication controls access to the XR visualization interface. In the transaction sublayer, ECDSA cryptographic signatures on all IoT sensor data prevent injection attacks where

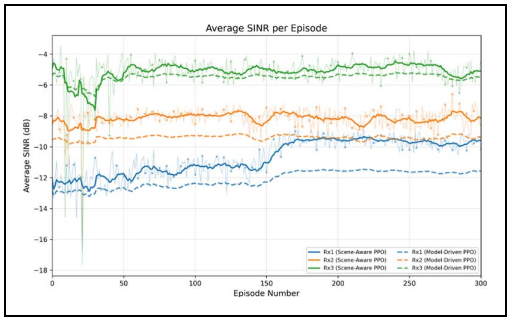

FIG. 4. Average SINR per episode during RL training. Solid lines: Scene-Aware PPO (SA-PPO); dashed lines: Model-Driven PPO (MD-PPO) baseline.

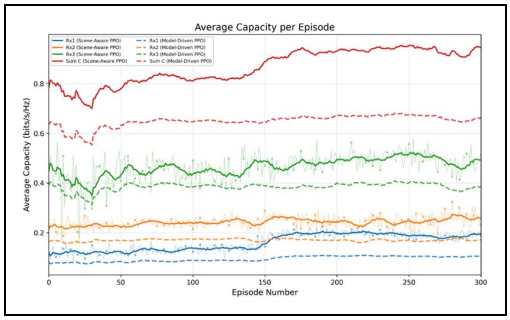

FIG. 5. Average capacity per episode during RL training. Solid lines: Scene-Aware PPO (SA-PPO); dashed lines: Model-Driven PPO (MD-PPO) baseline.

adversaries attempt to manipulate measurements that could mislead the AI agent.

D. Analysis of Training Results

The training dataset comprises 15,000 timesteps organized in 300 episodes of 50 steps each. Each timestep involves a full Sionna ray-tracing evaluation with 10^6 samples and a maximum reflection depth of 2. The total training generates 15,000 SINR and capacity observations across all three receivers.

The efficacy of the RL agent in optimizing the UAV's position is demonstrated by the training results, which are presented as plots of the average SINR and the resulting channel capacity over 300 training episodes. These metrics illustrate the agent's learning progress and the impact of its policy on network performance.

Fig. 4 shows the average SINR for each of the three ground receivers and indicates a successful learning trend. Initially, the SINR values are low and/or volatile, particularly for receiver 1 (blue line), which start at approximately -12 dB. As training progresses, the agent discovers more effective UAV positions. A significant improvement is visible around episode 150, after which the SINR for receiver 1 stabilizes at a higher level around -10 dB. This shows that the agent has successfully learned a policy that improves the quality of the link for the less advantageous user, without reducing the QoS of the other users. The direct consequence of this SINR optimization is shown in Fig. 5, which plots the average channel capacity for each receiver and their sum. The total network capacity (red line) exhibits a distinct learning curve that mirrors the SINR improvements, rising significantly and then stabilizing at a much higher level after episode 150. The apparent discrepancy between the pronounced SINR dips visible in Fig. 4 and their absence in Fig. 5 is explained by the logarithmic compression inherent in the Shannon capacity formula $C = \log_2(1 + \text{SINR})$. At low SINR values (below 0 dB), the capacity function compresses variations significantly—a 5 dB SINR change near -10 dB maps to only ~ 0.05 bits/s/Hz capacity change. Consequently, SINR fluctuations that appear pronounced on the dB scale translate to negligible variations in capacity.

We term our proposed approach *Scene-Aware PPO* (SA-PPO), as the agent receives channel observations from site-specific ray tracing within the 3D digital twin environment. As a baseline, we evaluate *Model-Driven PPO* (MD-PPO), which uses the same PPO architecture but receives channel observations from an analytical Rician fading model ($K = 10$ dB, typical for urban UAV-ground links at 25–30 m altitude) instead of ray tracing, with 10^5 Monte Carlo channel realizations per episode. Because the Rician model provides only statistical multipath information without site-specific geometry, the MD-PPO agent converges to a suboptimal policy compared to SA-PPO. All metrics reported below are averaged over the final 50 episodes (post-convergence).

SA-PPO outperforms MD-PPO across all three receivers. At convergence, SA-PPO achieves per-receiver SINR of -9.73 ± 0.38 dB (Rx1), -8.14 ± 0.64 dB (Rx2), and -5.08 ± 0.59 dB (Rx3), compared to MD-PPO's -11.65 ± 0.19 dB, -9.40 ± 0.32 dB, and -5.48 ± 0.29 dB, respectively—corresponding to SINR improvements of $+16.4\%$, $+13.4\%$, and $+7.2\%$. The most significant capacity improvement is observed for Rx1—the most disadvantaged receiver—where SA-PPO achieves 0.1872 ± 0.0144 bits/s/Hz, a $+79.6\%$ gain over the MD-PPO baseline (0.1042 ± 0.0044 bits/s/Hz). For Rx2, SA-PPO reaches 0.2557 ± 0.0276 bits/s/Hz versus MD-PPO's 0.1705 ± 0.0117 bits/s/Hz ($+49.9\%$), while Rx3 improves from 0.3852 ± 0.0223 to 0.4891 ± 0.0374 bits/s/Hz ($+27.0\%$). The higher standard deviations of SA-PPO (e.g., 0.38 dB vs. 0.19 dB for Rx1 SINR, 0.64 dB vs. 0.32 dB for Rx2) reflect the richer multipath dynamics captured by ray tracing.

The comparison between SA-PPO and MD-PPO illustrates the value of scene-aware channel observations. The substantial gap between the two agents demonstrates that the Sionna ray-tracing channel model captures constructive multipath propagation effects absent from statistical models, justifying the use of a physics-based digital twin for UAV positioning optimization. The baseline reference curves in Figs. 4 and 5 provide visual confirmation: solid lines (SA-PPO) and dashed curves (MD-PPO).

E. Reproducibility

All experiments were conducted on a workstation equipped with an NVIDIA GPU with TensorFlow-GPU support. The software stack comprises Python 3.x, Sionna [12] for ray-tracing and channel modeling, Stable-Baselines3 for the PPO implementation, Gymnasium for the RL environment interface, TensorFlow, and NumPy. The urban scene used is Sionna's built-in Munich environment. PPO hyperparameters are: learning rate 5×10^{-4}, discount factor $\gamma = 0.99$, 128 steps per rollout, batch size 32, clip range 0.2, entropy coefficient 0.005, and 3 training epochs per update. Training wall-clock time ranges from approximately 2 to 6 hours depending on the GPU model. All experiments use random seed 111 to ensure reproducibility. The source code and trained models are available from the authors upon request.

V. Conclusion & Future Directions

The framework presents an improvement in the way network operators, researchers, and developers simulate, emulate, and interact with wireless systems. Its modular design provides a tool for addressing modern systems. The use of DTs allows for real-time monitoring and optimization of performance. The networking layer enables ultra-low latency and reliable communication. AI-driven algorithms provide solutions for network management. IoT devices ensure that the system can adapt to real-world conditions. The blockchain provides secure and transparent logging of all network operations. The XR layer offers an interface for users to manage network data in a more convenient way. However, there are several areas for further research:

GenAI for Virtual World Creation: Using generative models to produce interactive 3D assets, users can rapidly prototype dynamic virtual worlds.

O-RAN Advancements: Future research can focus on: (i) High-Fidelity DTs for real-time simulation of complex behaviors. (ii) Specialized AI for RAN Intelligent Controllers (RICs). (iii) Advanced features through dApp implementations and smart contracts. (iv) Real-Time Synchronization between the infrastructure and its DT to support ultra-low-latency applications. (v) Semantic Communications to improve the efficiency of data exchange between physical and virtual elements.

Comprehensive Parametric Analysis: The next step is to conduct a parametric analysis of the UAV positioning use case. This involves varying key system parameters (such as the RL algorithm or IoT measurement distributions) to fully characterize the framework's performance.

References

[1] I. Lotfi, D. Niyato, S. Sun, D. I. Kim, and X. Shen, "Semantic information marketing in the metaverse: A learning-based contract theory framework," *IEEE J. Sel. Areas Commun.*, vol. 42, no. 3, pp. 710–723, Mar. 2024.

[2] L. U. Khan, Z. Han, D. Niyato, M. Guizani, and C. S. Hong, "Metaverse for wireless systems: Vision, enablers, architecture, and future directions,"*IEEE Wireless Commun.*, vol. 31, no. 4, pp. 245–251, Aug. 2023.

[3] O. Hashash, C. Chaccour, W. Saad, T. Yu, K. Sakaguchi, and M. Debbah, "The seven worlds and experiences of the wireless metaverse: Challenges and opportunities," *IEEE Commun. Surveys Tuts.*, vol. 63, no. 2, pp. 120–127, Feb. 2025.

[4] N. Sehad, L. Bariah, W. Hamidouche, H. Hellaoui, R. Jäntti, and M. Debbah, "Generative AI for immersive communication: The next frontier in internet-of-senses through 6G,"*IEEE Commun. Mag.*, vol. 63, no. 2, pp. 31–43, Feb. 2025.

[5] L. U. Khan, W. Saad, D. Niyato, Z. Han, and C. S. Hong, "Digital-twin-enabled 6G: Vision, architectural trends, and future directions," *IEEE Commun. Mag.*, vol. 60, no. 1, pp. 74–80, Jan. 2022.

[6] O-RAN Next Generation Research Group (nGRG), "dApps for real-time RAN control: Use cases and requirements," Contributed Research Rep. RR-2024-10, O-RAN ALLIANCE, Oct. 2024.

[7] M. Polese et al., "Colosseum: The open RAN digital twin," *IEEE Commun. Mag.*, vol. 5, no. 1, pp. 5452–5466, 2024.

[8] S. Jiang, Q. Qu, X. Pan, A. K. Agrawal, R. Newcombe, and A. Alkhateeb, "Learnable wireless digital twins: Reconstructing electromagnetic field with neural representations," *IEEE Open J. Commun. Soc.*, vol. 6, pp. 1568–1590, 2025.

[9] S. Alikhani, G. Charan, and A. Alkhateeb, "Large Wireless Model (LWM): A foundation model for wireless channels," Jan. 2024, *arXiv:2401.17115*.

[10] B. Yang, Y. Zhang, J. Wang, and H. Li, "Diffusion models as network optimizers: Explorations and analysis," Nov. 2024, *arXiv:2411.00453*.

[11] H. Zou et al., "TelecomGPT: A Framework to build telecom-specific large language models," Jul. 2024, *arXiv:2407.09424*.

[12] J. Hoydis et al., "Sionna: An open-source library for next-generation physical layer research," Mar. 2023, *arXiv:2203.11854*.

[13] Z. Lin, T. Wang, L. Shi, S. Zhang, and B. Cao, "Decentralized Physical Infrastructure Networks (DePIN): Challenges and opportunities," *IEEE Netw.*, vol. 39, no. 2, pp. 91–99, Mar. 2025.

[14] Y. Zou et al., "A survey of fault tolerant consensus in wireless networks," *High-Confidence Comput.*, vol. 4, no. 2, Jun. 2024, Art. no. 100202, doi: 10.1016/j.hcc.2024.100202.

[15] S. He, K. Shi, C. Liu, B. Guo, J. Chen, and Z. Shi, "Collaborative sensing in Internet of Things: A comprehensive survey," *IEEE Commun. Surveys Tuts.*, vol. 24, no. 3, pp. 1435–1474, 3rd Quart. 2022, doi: 10.1109/COMST.2022.3187138.